ادامه مقایسه مجموعه (Set Comparison)

- **بند "all":**

تعریف بند "all" به صورت زیر است:

F<comp>all r⇔∀t∈r (F<comp>t)F <comp> \text{all } r \Leftrightarrow \forall t \in r \text{ (} F <comp> t \text{)}F<comp>all r⇔∀t∈r (F<comp>t)

مثال‌ها:

(5<all [056])=false (5 < \text{all } \begin{bmatrix} 0 \\ 5 \\ 6 \end{bmatrix}) = \text{false} (5<all 056)=false

(5<all [610])=true (5 < \text{all } \begin{bmatrix} 6 \\ 10 \end{bmatrix}) = \text{true} (5<all [610])=true

(5=all [45])=false (5 = \text{all } \begin{bmatrix} 4 \\ 5 \end{bmatrix}) = \text{false} (5=all [45])=false

(5≠all [46])=true (since 5≠4 and 5≠6) (5 \neq \text{all } \begin{bmatrix} 4 \\ 6 \end{bmatrix}) = \text{true} \text{ (since } 5 \neq 4 \text{ and } 5 \neq 6 \text{)} (5□=all [46])=true (since 5□=4 and 5□=6)

نکته: not≡(all≠) in(\neq \text{all}) \equiv \text{not in} اما not≡(all=□) in(= \text{all})≢(all=) .in≡□(all=)\not\equiv \text{in}.

مثال کاربردی: یافتن نام تمام مدرسینی که حقوق آن‌ها از حقوق **تمامی** مدرسین دپارتمان زیست‌شناسی (Biology) بیشتر است.

```
sql
    select name
    from instructor
    where salary > all (select salary
                        from instructor
                        where dept_name = 'Biology');
```

- **بند "some":**

تعریف بند "some" به صورت زیر است (<comp><comp><comp> می‌تواند <,≥,<,=,≠,> \leq, <, =, \neq,>,≥,<,=,□= باشد):

F<comp>some r⇔∃t∈r such that (F<comp>t)F <comp> \text{some } r \Leftrightarrow \exists t \in r \text{ such that (} F <comp> t \text{)}F<comp>some r⇔∃t∈r such that (F<comp>t)

مثال‌ها (عبارت some>5text{some}\ > 5some>5 یعنی: عدد ۵ از حداقل یک تاپل در رابطه کوچکتر باشد):

(5<some [056])=true (5 < \text{some } \begin{bmatrix} 0 \\ 5 \\ 6 \end{bmatrix}) = \text{true}
(5<some 056)=true

(5<some [05])=false (5 < \text{some } \begin{bmatrix} 0 \\ 5 \end{bmatrix}) = \text{false}
(5<some [05])=false

(5=some [05])=true (5 = \text{some } \begin{bmatrix} 0 \\ 5 \end{bmatrix}) = \text{true}
(5=some [05])=true

(5≠some [05])=true (since 0≠5) (5 \neq \text{some } \begin{bmatrix} 0 \\ 5 \end{bmatrix}) = \text{true} \text{ (since } 0 \neq 5 \text{)} (5□=some [05])=true (since 0□=5)

نکته: (=some)≡in(= \text{some}) \equiv \text{in}. اما (≠some)≡not in(\neq \text{some}) \not\equiv \text{not in}.

مثال کاربردی: یافتن نام مدرسینی که حقوقشان بیشتر از حقوق **حداقل یک** مدرس در دپارتمان زیست‌شناسی (Biology) است.

این کوئری با استفاده از `some >` به شکل زیر نوشته می‌شود:

```
sql
    select name
    from instructor
    where salary > some (select salary
                         from instructor
                         where dept_name = 'Biology');
```

(معادل همین کوئری را قبلاً با پیوند (Join) جداول دیده بودیم).

تست برای روابط خالی (Test for Empty Relations)

- ساختار `exists` اگر زیرپرس‌وجوی آرگومان آن خالی نباشد، مقدار `true` برمی‌گرداند.
- exists r \Leftrightarrow r \neq \emptyset exists\ r text{exists } r \Leftrightarrow r □=∅
- not exists r \Leftrightarrow r = \emptyset not exists r text{not exists } r \Leftrightarrow r = ∅
- **استفاده از بند "exists":**

روش دیگری برای مشخص کردن این پرس‌وجو: "یافتن تمام دروسی که هم در پاییز ۲۰۱۷ و هم در بهار ۲۰۱۸ ارائه شده‌اند"

```
sql
    select course_id
    from section as S
    where semester = 'Fall' and year = 2017 and
          exists (select *
                  from section as T
```

```
                where semester = 'Spring' and year = 2018
                and S.course_id = T.course_id);
```

- نام همبستگی (Correlation name): متغیر `s` در پرس‌وجوی بیرونی.
- زیرپرس‌وجوی همبسته (Correlated subquery): پرس‌وجوی درونی (که به `s` ارجاع می‌دهد).
- **استفاده از بند "not exists":**

یافتن تمام دانشجویانی که تمام دروس ارائه‌شده در دپارتمان زیست‌شناسی (Biology) را گذرانده‌اند.

- نکته: Y⊇X⇔∅=Y−YX - Y = \emptyset \Leftrightarrow X \subseteq YX⊇X⇔∅=Y−X
- نکته: نمی‌توان این پرس‌وجو را با استفاده از `= all` و انواع آن نوشت.

```
sql
    select distinct S.ID, S.name
    from student as S
    where not exists ( (select course_id
                        from course
                        where dept_name = 'Biology')
                       except
                       (select T.course_id
                        from takes as T
                        where S.ID = T.ID) );
```

- پرس‌وجوی تودرتوی اول تمام دروس ارائه‌شده در زیست‌شناسی را لیست می‌کند.
- پرس‌وجوی تودرتوی دوم تمام دروسی که یک دانشجوی خاص گذرانده را لیست می‌کند.

اگر حاصل تفاضل این دو (except) خالی باشد، یعنی دانشجو تمام آن دروس را گذرانده است.

تست عدم وجود تاپل‌های تکراری (Test for Absence of Duplicate Tuples)

- ساختار `unique` بررسی می‌کند که آیا یک زیرپرس‌وجو دارای تاپل‌های تکراری در نتیجه خود هست یا خیر.
- اگر زیرپرس‌وجوی داده شده فاقد موارد تکراری باشد، `unique` به `true` ارزیابی می‌شود.
- مثال: یافتن تمامی دروسی که در سال ۲۰۱۷ حداکثر یک بار ارائه شده‌اند.

```
sql
    select T.course_id
    from course as T
    where unique ( select R.course_id
                   from section as R
                   where T.course_id = R.course_id
                   and R.year = 2017);
```

زیرپرس‌وجوها در بند From (Subqueries in the From Clause)

- SQL اجازه می‌دهد تا از یک عبارت زیرپرس‌وجو در بند `from` استفاده شود.

- مثال: یافتن میانگین حقوق مدرسینِ آن دپارتمان‌هایی که میانگین حقوقشان بیشتر از ۴۲,۰۰۰ دلار است.

```
sql
    select dept_name, avg_salary
    from ( select dept_name, avg (salary) as avg_salary
           from instructor
           group by dept_name)
    where avg_salary > 42000;
```

- دقت کنید که با این روش نیازی به استفاده از بند `having` نداریم.
- روش دیگر برای نوشتن پرس‌وجوی بالا (نام‌گذاری جدول و ستون‌های نتیجه زیرپرس‌وجو):

```
sql
    select dept_name, avg_salary
    from ( select dept_name, avg (salary)
           from instructor
           group by dept_name)
           as dept_avg (dept_name, avg_salary)
    where avg_salary > 42000;
```

بند With (With Clause)

- بند `with` راهی برای تعریف یک رابطه موقت (Temporary relation) فراهم می‌کند که تعریف آن فقط برای پرس‌وجویی که `with` در آن قرار دارد در دسترس است.
- مثال: یافتن تمام دپارتمان‌هایی که بیشترین بودجه را دارند.

```
sql
    with max_budget (value) as
         (select max(budget)
          from department)
    select department.name
    from department, max_budget
    where department.budget = max_budget.value;
```

- **پرس‌وجوهای پیچیده با استفاده از بند With:**

مثال: یافتن تمام دپارتمان‌هایی که مجموع حقوقشان از میانگین مجموع حقوق تمام دپارتمان‌ها بیشتر است.

```
sql
    with dept_total (dept_name, value) as
         (select dept_name, sum(salary)
          from instructor
          group by dept_name),
         dept_total_avg(value) as
         (select avg(value)
          from dept_total)
    select dept_name
```

```
    from dept_total, dept_total_avg
    where dept_total.value > dept_total_avg.value;
```

زیرپرس‌وجوی اسکالر (Scalar Subquery)

- زیرپرس‌وجوی اسکالر زیرپرس‌وجویی است که در جایی استفاده می‌شود که انتظار می‌رود یک مقدار واحد برگردانده شود.
- مثال: لیست کردن تمام دپارتمان‌ها به همراه تعداد مدرسین هر دپارتمان.

```
sql
    select dept_name,
           ( select count(*)
             from instructor
             where department.dept_name = instructor.dept_name)
             as num_instructors
    from department;
```

- اگر زیرپرس‌وجو بیش از یک تاپل نتیجه را برگرداند، خطای زمان اجرا (Runtime error) رخ می‌دهد.

اصلاح پایگاه داده (Modification of the Database)

شامل:

- حذف تاپل‌ها از یک رابطه معین (Deletion)
- درج تاپل‌های جدید در یک رابطه معین (Insertion)
- به‌روزرسانی مقادیر برخی تاپل‌ها در یک رابطه معین (Updating)

حذف (Deletion)

- حذف تمام مدرسین: `delete from instructor`
- حذف تمام مدرسین دپارتمان Finance:

```
sql
    delete from instructor
    where dept_name= 'Finance';
```

- حذف تمام تاپل‌های جدول instructor برای آن دسته از مدرسینی که با دپارتمانی در ساختمان ‘Watson’ مرتبط هستند:

```
sql
    delete from instructor
    where dept_name in (select dept_name
                        from department
                        where building = 'Watson');
```

- حذف تمام مدرسینی که حقوق آن‌ها کمتر از میانگین حقوق مدرسین است.
- مشکل: همانطور که تاپل‌ها را از `instructor` حذف می‌کنیم، میانگین حقوق تغییر می‌کند.
- راه‌حل مورد استفاده در SQL:

1. ابتدا میانگین حقوق را محاسبه کرده و تمام تاپل‌هایی که باید حذف شوند را پیدا می‌کند.
2. سپس تمام تاپل‌های یافت‌شده در مرحله قبل را حذف می‌کند (بدون محاسبه مجدد میانگین یا آزمایش مجدد تاپل‌ها).

```
sql
    delete from instructor
    where salary < (select avg (salary)
                    from instructor);
```

درج (Insertion)

- افزودن یک تاپل جدید به دوره (course):

```
sql
    insert into course
    values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

یا به صورت معادل (با ذکر صریح نام ستون‌ها):

```
sql
    insert into course (course_id, title, dept_name, credits)
    values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

- افزودن یک تاپل جدید به دانشجو (student) با تنظیم `tot_creds` به مقدار `null`:

```
sql
    insert into student
    values ('3003', 'Green', 'Finance', null);
```

- هر دانشجویی در دپارتمان ‘Music’ که بیش از ۱۴۴ واحد گذرانده است را با حقوق ۱۸,۰۰۰ به عنوان مدرس در همان دپارتمان ثبت کنید:

```
sql
    insert into instructor
    select ID, name, dept_name, 18000
    from student
    where dept_name = 'Music' and total_cred > 144;
```

نکته: دستور `select-from-where` قبل از اینکه هر یک از نتایج آن در رابطه درج شود، به طور کامل ارزیابی می‌شود. در غیر این صورت پرس‌وجوهایی مانند `insert into table1 select * from table1` باعث ایجاد مشکل می‌شوند.

به‌روزرسانی (Updates)

- افزایش ۵ درصدی حقوق تمامی مدرسین:

```
sql
    update instructor
    set salary = salary * 1.05
```

- افزایش ۵ درصدی حقوق آن دسته از مدرسینی که کمتر از ۷۰,۰۰۰ درآمد دارند:

```
sql
    update instructor
    set salary = salary * 1.05
    where salary < 70000;
```

- افزایش ۵ درصدی حقوق مدرسینی که حقوقشان کمتر از میانگین است:

```
sql
    update instructor
    set salary = salary * 1.05
    where salary < (select avg (salary)
                    from instructor);
```

- افزایش ۳ درصدی حقوق مدرسینی که حقوقشان بیش از ۱۰۰,۰۰۰ است و ۵ درصدی برای بقیه.
- با نوشتن دو دستور آپدیت:

```
sql
        update instructor
        set salary = salary * 1.03
        where salary > 100000;

        update instructor
        set salary = salary * 1.05
        where salary <= 100000;
```

ترتیب مهم است!

- بهتر است این کار با استفاده از دستور `case` انجام شود:

```
sql
        update instructor
        set salary = case
            when salary <= 100000 then salary * 1.05
            else salary * 1.03
            end
```

- **به‌روزرسانی با زیرپرس‌وجوهای اسکالر:**

محاسبه مجدد و به‌روزرسانی مقدار tot_creds برای تمامی دانشجویان:

```
sql
    update student S
    set tot_cred = (select sum(credits)
                    from takes, course
                    where takes.course_id = course.course_id and
                          S.ID= takes.ID and
                          takes.grade <> 'F' and
                          takes.grade is not null);
```

- این دستور tot_creds را برای دانشجویانی که هیچ دوره‌ای نگذرانده‌اند، null تنظیم می‌کند.
- برای جلوگیری از این مشکل، به جای sum(credits) می‌توان از این ساختار استفاده کرد:

```
sql
        case
            when sum(credits) is not null then sum(credits)
            else 0
        end
```